

MỘT SỐ DẠNG BÀI TẬP CƠ BẢN

BÀI 1: LỚP BÀI TOÁN CÁI TÚI

Trong siêu thị có n đồ vật ($n \leq 1000$), đồ vật thứ i có trọng lượng là $W[i] \leq 1000$ và giá trị $V[i] \leq 1000$. Một tên trộm đột nhập vào siêu thị, tên trộm mang theo một cái túi có thể mang được tối đa trọng lượng M ($M \leq 1000$). Hỏi tên trộm sẽ lấy đi những đồ vật nào để được tổng giá trị lớn nhất.

Giải quyết bài toán trong các trường hợp sau:

- Mỗi vật chỉ được chọn một lần.
- Mỗi vật được chọn nhiều lần (không hạn chế số lần)

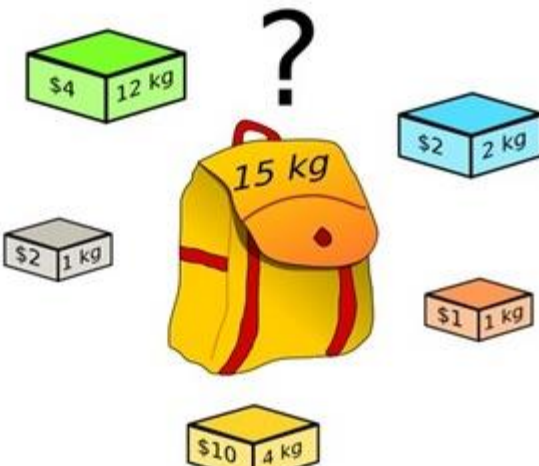
InputData: file văn bản **Bag.inp**

- Dòng 1: n, M cách nhau ít nhất một dấu cách
- n dòng tiếp theo: Mỗi dòng gồm 2 số W_i, V_i , là chi phí và giá trị đồ vật thứ i .

OutputData: file văn bản **bag.out**: Ghi giá trị lớn nhất tên trộm có thể lấy

Example

Input	Output
5 15 12 4 2 2 1 1 1 2 4 10	15



Hướng dẫn giải

Trường hợp mỗi vật được chọn 1 lần

Ta nhận thấy rằng: Giá trị của cái túi phụ thuộc vào 2 yếu tố: Có bao nhiêu vật đang được xét và trọng lượng còn lại cái túi có thể chứa được, do vậy chúng ta có 2 đại lượng biến thiên. Cho nên hàm mục tiêu sẽ phụ thuộc vào hai đại lượng biến thiên.

Do vậy bảng phương án của chúng ta sẽ là bảng 2 chiều.

Gọi $F[i, j]$ là tổng giá trị lớn nhất của cái túi khi xét từ vật 1 đến vật i và trọng của cái túi chưa vượt quá j . Với giới hạn j , việc chọn tối ưu trong số các vật $\{1, 2, \dots, i-1, i\}$ để có giá trị lớn nhất sẽ có hai khả năng:

- Nếu không chọn vật thứ i thì $F[i,j]$ là giá trị lớn nhất có thể chọn trong số các vật $\{1,2,\dots,i-1\}$ với giới hạn trọng lượng là j , tức là: $F[i,j] := F[i-1,j]$.
- Nếu có chọn vật thứ i (phải thỏa điều kiện $W[i] \leq j$) thì $F[i,j]$ bằng giá trị vật thứ i là $V[i]$ cộng với giá trị lớn nhất có thể có được bằng cách chọn trong số các vật $\{1,2,\dots,i-1\}$ với giới hạn trọng lượng $j-W[i]$ tức là về mặt giá trị thu được:

$$F[i,j] := V[i] + F[i-1,j-W[i]]$$

Vậy chúng ta phải xem xét xem nếu chọn vật i hay không chọn vật i thì sẽ tốt hơn. Từ đó chúng ta có công thức truy hồi như sau.

- $F[0,j] = 0$ (hiển nhiên) – Bài toán con nhỏ nhất.
- $F[i,j] = \max(F[i-1,j], V[i] + F[i-1,j-W[i]])$

Trường hợp mỗi vật được chọn nhiều lần: Tương tự như suy luận ở trên ta xét:

Nếu không chọn vật thứ i thì $F[i,j]$ là giá trị lớn nhất có thể chọn trong số các vật $\{1,2,\dots,i-1\}$ với giới hạn trọng lượng là j , tức là: $F[i,j] := F[i-1,j]$

Nếu có chọn vật thứ i (phải thỏa điều kiện $W[i] \leq j$) thì $F[i,j]$ bằng giá trị vật thứ i là $V[i]$ cộng với giá trị lớn nhất có thể có được bằng cách chọn trong số các vật $\{1,2,\dots,i\}$ (vì vật i vẫn có thể được chọn tiếp) với giới hạn trọng lượng $j-W[i]$ tức là về mặt giá trị thu được: $F[i,j] := V[i] + F[i,j-W[i]]$

Do vậy chúng ta có **công thức truy hồi** như sau:

- $F[0,j] = 0$ (hiển nhiên) – Bài toán con nhỏ nhất.
- $F[i,j] = \max(F[i-1,j], V[i] + F[i,j-W[i]])$

3. Bảng phương án

Ta xây dựng bảng phương án dựa trên công thức truy hồi trên. Để kiểm tra kết quả có chính xác hay không (nếu không chính xác chúng ta xây dựng lại hàm mục tiêu).

Thông qua cách xây dựng hàm mục tiêu và bảng phương án chúng ta sẽ định hướng việc truy vết.

Example (trường hợp mỗi vật chỉ được chọn 1 lần)

Bảng phương án:

NM	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2	0	0	2	2	2	2	2	2	2	2	2	2	4	4	6	6
3	0	1	2	3	3	3	3	3	3	3	3	3	4	5	6	7
4	0	2	3	4	5	5	5	5	5	5	5	5	5	6	7	8
5	0	2	3	4	10	12	13	14	15	15	15	15	15	15	15	15

Vậy chúng ta có thể chọn vật 2, 3, 4, 5.

Example (trường hợp mỗi vật được chọn nhiều lần)

Input	Output
5 15 12 4 2 2 1 1 1 2 4 10	36

Bảng phương án:

N/M	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2	0	0	2	2	4	4	6	6	8	8	10	10	12	12	14	14
3	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
4	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30
5	0	2	4	6	10	12	14	16	20	22	24	26	30	32	34	36

Chúng ta có thể chọn vật 4 (3 lần) và vật 5 (3 lần).

4. Truy vết

Trường hợp 1: Trong bảng *phương án* $F[n,m]$ chính là giá trị lớn nhất thu được khi chọn trong cả n vật với giới hạn trọng lượng là M .

Nếu $f[n,M]=f[n-1,M]$ thì tức là không chọn vật thứ n , ta truy về $f[n-1,M]$. Còn nếu $f[n,M] \neq f[n-1,M]$ thì ta thông báo rằng phép chọn tối ưu có chọn vật thứ n và truy về $f[n-1,M-W_n]$.

Trường hợp 2: Trong bảng *phương án* $F[n,m]$ chính là giá trị lớn nhất thu được khi chọn trong cả n vật với giới hạn trọng lượng là M .

Nếu $f[n,M]=f[n-1,M]$ thì tức là không chọn vật thứ n , ta truy về $f[n-1,M]$. Còn nếu $f[n,M] \neq f[n-1,M]$ thì ta thông báo rằng phép chọn tối ưu có chọn vật thứ n và truy về $f[n,M-W_n]$.

5. Cài đặt

Program Bag;

Const

limit = 1000;

Var

V,W: Array[1..limit] of integer;

F: Array[0..limit,0..limit] of integer;

n,M,i,j: integer;

fi,fo: text;

{-----}

Procedure inputdata;

Begin

```

        assign(fi,'bag.inp');
        reset(fi);
        readln(fi,n,m);
        for i:=1 to n do readln(fi,w[i],v[i]);
        close(fi);
    End;
}-----}
Procedure process;
    Begin
        For i:=1 to n do f[i,0]:=0;
        For j:=1 to m do f[0,j]:=0;
        for i:=1 to n do
            for j:=1 to m do
                if w[i]<=j then F[i,j]:=max(F[i-1,j],F[i-1,j-w[i]]+v[i])
                else    F[i,j]:=F[i-1,j];
            End;
        End;
    }-----}
Function max(a,b:integer):integer;
    Begin
        if a>b then max:=a
        else max:=b;
    End;
}-----}
Procedure outputdata;
    Begin
        assign(fo,'bag.out');
        rewrite(fo);
        write(fo,F[n,m]);
        close(fo);
    End;
}-----}
Begin
    inputdata;
    process;
    outputdata;
End.

```

BÀI 2: DÃY CON TĂNG DÀI NHẤT

Cho dãy số nguyên $A = a_1, a_2, \dots, a_n$. ($n \leq 10000$, $-10000 \leq a_i \leq 10000$). **Một dãy con của A là một cách chọn ra trong A một số phần tử giữ nguyên thứ tự.** Như vậy A có 2^n dãy con.

Yêu cầu: Tìm dãy con đơn điệu tăng của A có độ dài lớn nhất.

Ví dụ: $A = (1, 2, 3, 4, 9, 10, 5, 6, 7, 8)$. Dãy con đơn điệu tăng dài nhất là: $(1, 2, 3, 4, 5, 6, 7, 8)$.

- Dữ liệu (**Input**) vào từ file văn bản INCSEQ.INP

- Dòng 1: Chứa số n
- Dòng 2: Chứa n số $a_1, a_2, \dots, a_n$ cách nhau ít nhất một dấu cách

- Kết quả (**Output**) ghi ra file văn bản INCSEQ.OUT

- Dòng 1: Ghi độ dài dãy con tìm được
- Các dòng tiếp: ghi dãy con tìm được và chỉ số những phần tử được chọn vào dãy con đó.

Ví dụ:

INCSEQ.INP	INCSEQ.OUT
11	8
1 2 3 8 9 4 5 6	a[1] = 1
20 9 10	a[2] = 2
	a[3] = 3
	a[6] = 4
	a[7] = 5
	a[8] = 6
	a[10] = 9
	a[11] = 10

Cách giải:

Bổ sung vào A hai phần tử: $a_0 = -\infty$ ($-\text{maxint}$) và $a_{n+1} = +\infty$ (maxint). **Khi đó dãy con đơn điệu tăng dài nhất chắc chắn sẽ bắt đầu từ a_0 và kết thúc ở a_{n+1} .**

Với mọi i : $0 \leq i \leq n + 1$. Ta sẽ tính $L[i]$ = độ dài dãy con đơn điệu tăng dài nhất bắt đầu tại a_i .

1. Cơ sở quy hoạch động (bài toán nhỏ nhất):

$L[n+1]$ = Độ dài dãy con đơn điệu tăng dài nhất bắt đầu tại $a_{n+1} = +\infty$. Dãy con này chỉ gồm mỗi một phần tử ($+\infty$) nên $L[n+1]=1$.

2. Công thức truy hồi:

Giả sử với i từ n đến 0 , ta cần tính $L[i]$: độ dài dãy con tăng dài nhất bắt đầu tại a_i .

$L[i]$ được tính trong điều kiện $L[i+1], L[i+2], \dots, L[n+1]$ đã biết:

Dãy con đơn điệu tăng dài nhất bắt đầu từ a_i sẽ được thành lập bằng cách lấy a_i ghép vào đầu một trong số những dãy con đơn điệu tăng dài nhất bắt đầu tại vị trí a_j đứng sau a_i . Ta sẽ chọn dãy nào để ghép a_i vào đầu? Tất nhiên là chỉ được ghép a_i vào đầu những dãy con bắt đầu tại a_j nào đó lớn hơn a_i (để đảm bảo tính tăng) và dĩ nhiên ta sẽ chọn dãy dài nhất để ghép a_i vào đầu (để đảm bảo tính dài nhất). Vậy $L[i]$ được

tính như sau: Xét tất cả các chỉ số j trong khoảng từ $i + 1$ đến $n + 1$ mà $a_j > a_i$, chọn ra chỉ số $j_{\max}$ có $L[j_{\max}]$ lớn nhất. Đặt $L[i] := L[j_{\max}] + 1$.

3. Truy vết

Tại bước xây dựng dãy L , mỗi khi tính $L[i] = L[j_{\max}] + 1$, ta đặt $T[i] = j_{\max}$. Để lưu lại rằng: Dãy con dài nhất bắt đầu tại a_i sẽ có phần tử thứ hai kế tiếp là $a_{j_{\max}}$.

Sau khi tính xong hay dãy L và T , ta bắt đầu từ 0.

$T[0]$ là phần tử đầu tiên được chọn,

$T[T[0]]$ là phần tử thứ hai được chọn,

$T[T[T[0]]]$ là phần tử thứ ba được chọn ... Quá trình truy vết có thể diễn tả như sau:

$i := T[0];$

while $i < n + 1$ **do** {Chừng nào chưa duyệt đến số $a_{n+1} = +\infty$ ở cuối}

begin

 <Thông báo chọn a_i >

$i := T[i];$

end;

Ví dụ: với $A = (5, 2, 3, 4, 9, 10, 5, 6, 7, 8)$. Hai dãy L và T sau khi tính sẽ là:

```
program LongestSubSequence;
```

```
const max = 10000;
```

```
var a, L, T: array[0..max + 1] of Integer;
```

```
  n: Word;
```

```
procedure Enter; {Nhập dữ liệu từ thiết bị nhập chuẩn theo đúng khuôn dạng Input}
```

```
var i: Word;
```

```
begin
```

```
  ReadLn(n);
```

```
  for i := 1 to n do Read(a[i]);
```

```
end;
```

```
procedure Optimize; {Quy hoạch động}
```

```
var
```

```
  i, j, jmax: Word;
```

```
begin
```

```
  a[0] := -32768; a[n + 1] := 32767; {Thêm hai phần tử canh hai đầu dãy a}
```

```
  L[n + 1] := 1; {Điền cơ sở quy hoạch động vào bảng phương án}
```

```
  for i := n downto 0 do {Tính bảng phương án}
```

```
    begin
```

```
      {Chọn trong các chỉ số  $j$  đứng sau  $i$  thoả mãn  $a_j > a_i$  ra chỉ số  $j_{\max}$  có  $L[j_{\max}]$  lớn nhất}
```

```

jmax := n + 1;
for j := i + 1 to n + 1 do
  if (a[j] > a[i]) and (L[j] > L[jmax]) then jmax := j;
  L[i] := L[jmax] + 1; {Lưu độ dài dãy con tăng dài nhất bắt đầu tại ai}
  T[i] := jmax; {Lưu vết: phần tử đứng liền sau ai trong dãy con tăng dài nhất đó là
ajmax}
end;
WriteLn(L[0] - 2); {Chiều dài dãy con tăng dài nhất}
i := T[0]; {Bắt đầu truy vết tìm nghiệm}
while i <> n + 1 do
  begin
    WriteLn('a[' , i , ']' = ' , a[i]);    i := T[i];
  end;
end;
begin
  {Định nghĩa lại thiết bị nhập/xuất chuẩn}
  Assign(Input, 'INCSEQ.INP'); Reset(Input);
  Assign(Output, 'INCSEQ.OUT'); Rewrite(Output);
  Enter;
  Optimize;
  Close(Input); Close(Output);
end.

```

có 1 con ngựa bị buộc vào cổ 1 đoạn dây thừng dài 5m. Mà trong khi đó bãi cỏ chỉ cách chỗ con ngựa 10m. để các bạn biết con ngựa ra bãi cỏ đây bằng cách nào?

1, Một con ốc sên bò lên cây cao 20 m, buổi sáng nó bò lên được 5m, buổi tối nó tụt xuống 4m. Hỏi mất bao nhiêu ngày thì con ốc sên bò lên được ngọn cây???

16 ngày, 15 ngày=15m, cộng ngày cuối cùng 5m, tới ngọn

2, Trong một cái hồ, có một bông sen . Cứ sau một ngày trôi qua , bông sen sẽ nở gấp đôi, cứ thế, chỉ trong vòng 30 ngày là sen nở đầy hồ. Hỏi để sen nở nửa hồ thì cần bao nhiêu ngày

có 1 chú ốc sên bò lên 1 bức tường cao 12m ,ban ngày nó bò được 3m, ban đêm tụt xuống còn 1m hỏi? bao nhiêu ngày .bao nhiêu đêm chú ốc sên bò lên được bức tường đó.

<http://www.lenquyenpc.net/2012/02/bai-toan-day-con-on-ieu-tang-dai-nhat.html>

Dãy con dài nhất có tổng chia hết cho K(QBSEQ)

```
Const MaxN=1000;
```

```
MaxK=50;
```

```
Var
```

```
A:Array[1..MaxN] of integer;
```

```
F:Array[0..MaxN,0..MaxK-1] of integer;
```

```
N,K:integer;
```

```
Procedure Input;
```

```
Var i:integer;
```

```
Begin
```

```
Readln(N,K);
```

```
For i:=1 to N do
```

```
Begin
```

```
Read(A[i]);
```

```
End;
```

```
End;
```

```
Function Sodu(X,Y:integer):integer;
```

```
Var Du:integer;
```

```
Begin
```

```
Du:=(X-Y) mod K;
```

```
If Du>=0 then Sodu:=Du
```

```
Else Sodu:=Du+K;
```

```

End;

Procedure Xuli;
Var i,j:integer;
Begin
F[0,0]:=0;
For j:=1 to K-1 do F[0,j]:=MaxK;
For i:=1 to N do
For j:=0 to K-1 do
If F[i-1,j]>=F[i-1,Sodu(A[i],j)]+1 then F[i,j]:=F[i-1,Sodu(A[i],j)]+1
Else F[i,j]:=F[i-1,j];
End;

```

```

Procedure Output;
Var i,S,M:integer;
Begin
S:=0;
For i:=1 to N do S:=S+A[i];
M:=S mod K;
Writeln(N-F[N,M]);
End;

```

```

BEGIN
Input;
Xuli;
Output;
END.

```

=====

Ta gọi một dãy chia hết hoàn toàn là dãy $a_1, a_2, \dots, a_n$ và $a[j]$ chia hết $a[i]$ với $j < i$. Một dãy con của một dãy là một dãy được thiết lập bằng cách xoá 1 số p/tử nào đó trong dãy.

vd :

3 7 11 3 : Là dãy không chia hết .

3 15 60 720 : là dãy chia hết .

Yêu cầu : Cho một dãy các số nguyên $a_1, a_2, \dots, a_n$. Tìm dãy con chia hết hoàn toàn dài nhất .

Dữ liệu vào : CHIAHET.INP

- Dòng 1 : Ghi số n .
- Dòng 2 : Ghi $a_1, a_2, \dots, a_n$.

Dữ liệu ra : CHIAHET.OUT

- Dòng 1 : Ghi độ dài dãy chia hết hoàn toàn .
- Dòng 2 : Ghi dãy ra (nếu có nhiều dãy dài nhất có độ dài bằng nhau thì ghi ra bất kì một dãy trong chúng) .

vd : CHIAHET.INP

```

9
2 3 7 8 14 39 145 76 320

```

CHIAHET.OUT

```

3
2 8 320
=====

```

BÀI 3: DÃY CON CÓ TỔNG CHIA HẾT CHO K

Cho một dãy gồm n ($1 \leq n \leq 1000$) số nguyên dương $A_1, A_2, \dots, A_n$ và số nguyên dương k ($k \leq 50$). Hãy tìm dãy con gồm nhiều phần tử nhất của dãy đã cho sao cho tổng các phần tử của dãy con này chia hết cho k.

Input: file văn bản SUBSEQ.INP

- Dòng 1: Chứa số n
- Dòng 2: Chứa n số $A_1, A_2, \dots, A_n$ cách nhau ít nhất một dấu cách

Output: file văn bản SUBSEQ.OUT

- Dòng 1: Ghi độ dài dãy con tìm được
- Các dòng tiếp: Ghi các phần tử được chọn vào dãy con
- Dòng cuối: Ghi tổng các phần tử của dãy con đó.

SUBSEQ.INP	SUBSEQ.OUT
10 5	8
1 6 11 5 10 15 20 2 4 9	a[10] = 9
	a[9] = 4
	a[7] = 20
	a[6] = 15
	a[5] = 10
	a[4] = 5
	a[3] = 11
	a[2] = 6
	Sum = 80

3.4.1. Cách giải 1

Đề bài yêu cầu chọn ra một số tối đa các phần tử trong dãy A để được một dãy có tổng chia hết cho k, ta có thể giải bài toán bằng phương pháp duyệt tổ hợp bằng quay lui có đánh giá nhánh cận nhằm giảm bớt chi phí trong kỹ thuật vét cạn. Dưới đây ta trình bày phương pháp quy hoạch động:

Nhận xét 1: Không ảnh hưởng đến kết quả cuối cùng, ta có thể đặt:

$$A_i := A_i \bmod k \text{ với } \forall i: 1 \leq i \leq n$$

Nhận xét 2: Gọi S là tổng các phần tử trong mảng A , ta có thể thay đổi cách tiếp cận bài toán: thay vì tìm xem phải chọn ra một số tối đa những phần tử để có tổng chia hết cho k , ta sẽ chọn ra một số tối thiểu các phần tử có tổng đồng dư với S theo modul k . Khi đó chỉ cần loại bỏ những phần tử này thì những phần tử còn lại sẽ là kết quả.

Nhận xét 3: Số phần tử tối thiểu cần loại bỏ bao giờ cũng nhỏ hơn k

Thật vậy, giả sử số phần tử ít nhất cần loại bỏ là m và các phần tử cần loại bỏ là $A_{i_1}, A_{i_2}, \dots, A_{i_m}$. Các phần tử này có tổng đồng dư với S theo mô-đun k . Xét các dãy sau

Dãy 0 := () = Dãy rỗng (Tổng $\equiv 0 \pmod{k}$)

Dãy 1 := (A_{i_1})

Dãy 2 := (A_{i_1}, A_{i_2})

Dãy 3 := ($A_{i_1}, A_{i_2}, A_{i_3}$)

... ..

Dãy m := ($A_{i_1}, A_{i_2}, \dots, A_{i_m}$)

Như vậy có $m + 1$ dãy, nếu $m \geq k$ thì theo nguyên lý Dirichlet sẽ tồn tại hai dãy có tổng đồng dư theo mô-đun k . Giả sử đó là hai dãy:

$$A_{i_1} + A_{i_2} + \dots + A_{i_p} \equiv A_{i_1} + A_{i_2} + \dots + A_{i_p} + A_{i_{p+1}} + \dots + A_{i_q} \pmod{k}$$

Suy ra $A_{i_{p+1}} + \dots + A_{i_q}$ chia hết cho k . Vậy ta có thể xoá hết các phần tử này trong dãy đã chọn mà vẫn được một dãy có tổng đồng dư với S theo modul k , mâu thuẫn với giả thiết là dãy đã chọn có số phần tử tối thiểu.

Công thức truy hồi:

Nếu ta gọi $F[i, t]$ là số phần tử tối thiểu phải chọn trong dãy $A_1, A_2, \dots, A_i$ để có tổng chia k dư t . Nếu không có phương án chọn ta coi $F[i, t] = +\infty$. Khi đó $F[i, t]$ được tính qua công thức truy hồi sau:

Nếu trong dãy trên không phải chọn A_i thì $F[i, t] = F[i - 1, t]$;

Nếu trong dãy trên phải chọn A_i thì $F[i, t] = 1 + F[i - 1, t - A_i]$ ($t - A_i$ ở đây hiểu là phép trừ trên các lớp đồng dư mod k . Ví dụ khi $k = 7$ thì $\overline{1 - 3} = 5$)

Từ trên suy ra $F[i, t] = \min(F[i - 1, t], 1 + F[i - 1, t - A_i])$.

Còn tất nhiên, cơ sở quy hoạch động: $F(0, 0) = 0$; $F(0, i) = +\infty$ (với $\forall i: 1 \leq i < k$).

Bảng phương án F có kích thước $[0..n, 0..k - 1]$ tối đa là 1001×50 phần tử kiểu Byte.

```

program SubSequence;
const
    InputFile = 'SUBSEQ.INP';
    OutputFile = 'SUBSEQ.OUT';
    maxN = 1000;
    maxK = 50;
var
    a: array[1..maxN] of Integer;
    f: array[0..maxN, 0..maxK - 1] of Byte;
    n, k: Integer;

procedure Enter;
var
    fi: Text;
    i: Integer;
begin
    Assign(fi, InputFile); Reset(fi);
    ReadLn(fi, n, k);
    for i := 1 to n do Read(fi, a[i]);
    Close(fi);
end;

function Sub(x, y: Integer): Integer; {Tính x - y (theo mod k)}
var
    tmp: Integer;
begin
    tmp := (x - y) mod k;
    if tmp >= 0 then Sub := tmp
    else Sub := tmp + k;
end;

procedure Optimize;
var
    i, t: Integer;
begin
    FillChar(f, SizeOf(f), $FF); {Khởi tạo các phần tử f[0, .] đều bằng 255 (+x)}
    f[0, 0] := 0; {Ngoại trừ f[0, 0] := 0}

```

```

for i := 1 to n do
  for t := 0 to k - 1 do {Tính f[i, t] := min (f[i - 1, t], f[i - 1, Sub(t, ai)] + 1}
    if f[i - 1, t] < f[i - 1, Sub(t, a[i])] + 1 then
      f[i, t] := f[i - 1, t]
    else
      f[i, t] := f[i - 1, Sub(t, a[i])] + 1;
end;

procedure Result;
var
  fo: Text;
  i, t: Integer;
  SumAll, Sum: LongInt;
begin
  SumAll := 0;
  for i := 1 to n do SumAll := SumAll + a[i];
  Assign(fo, OutputFile); Rewrite(fo);
  WriteLn(fo, n - f[n, SumAll mod k]); {n - số phần tử bỏ đi = số phần tử giữ lại}
  i := n; t := SumAll mod k;
  Sum := 0;
  for i := n downto 1 do
    if f[i, t] = f[i - 1, t] then {Nếu phương án tối ưu không bỏ ai, tức là có chọn ai}
      begin
        WriteLn(fo, 'a[' , i, ' ] = ' , a[i]);
        Sum := Sum + a[i];
      end
    else
      t := Sub(t, a[i]);
      WriteLn(fo, 'Sum = ' , Sum);
      Close(fo);
end;

begin
  Enter;
  Optimize;
  Result;
end.

```

Bài 4: "Dãy răng cưa" là dãy giảm tăng hoặc tăng giảm. Ví dụ: 1,4,2,7,3,8 hoặc 4,1,3,2,8,5,6 là dãy răng cưa. Nhập một dãy số nguyên, sau đó biến đổi nó để được một dãy răng cưa.

Viết chương trình nhập vào số nguyên N , thực hiện in lên màn hình các chữ số từ 1 --> $N*N$ theo hình xoắn ốc

VD: với $N=5$ ta có

```

1 2 3 4 5
16 17 18 19 6
15 24 25 20 7
14 23 22 21 8
13 12 11 10 9

```

```

uses crt;
var i,j,k,t,s,n,m: integer;
    a: array[1..50,1..50] of integer;

begin
  clrscr;
  write('Nhập n: ');
  readln(n);
  m:=n;
  i:=1; j:=0;
  s:=n; t:=1; k:=0;

```

```
for k:=1 to n*n do
begin
  case t of
    1 : j:=j+1;
    2 : i:=i+1;
    3 : j:=j-1;
    4 : i:=i-1;
  end;
  a[i,j]:=k;
  if k=s then
  begin
    n:=n-(t mod 2);
    t:=t+1;
    s:=s+n;
    if t=5 then t:=1;
  end;
end;
for i:=1 to m do
begin
  for j:= 1 to m do
    write(a[i,j]:3);
  writeln;
end;
readln
end.
```